

# Quiz 1 (Version 2)

CS 320: Concepts of Programming Languages

September 16, 2026

Name: Nathan Mull

BUID: 12345678

- You will have approximately 30 minutes to complete this quiz. Make sure to read every question; some are easier than others.
- Make very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).
- We will not look at any work on the pages marked *"This page is intentionally left blank."*. You should use these pages for scratch work.

# 1 OCaml Programming

(10 points) Implement the function

```
val muls : int -> int -> int list
```

so that `muls k n` is the list of all positive multiples of `k` at most `n` in increasing order. The behavior of the function is undefined if `k` is not positive.

```
let _ = assert (muls 3 14 = [3;6;9;12])
let _ = assert (muls 2 6 = [2;4;6])
let _ = assert (muls 51 3 = [])
```

You may not use any functions from the standard library except for arithmetic operators and list constructors (i.e., `[]` and `(::)`).

```
let mul k n =
  let rec go i =
    if i > n
    then []
    else i :: go (i+k)
  in go k
```

## 2 Types and Values

(10 points) Determine the type of the function `foo` and the value of `bar` below.

```
let rec foo l =  
  match l with  
  | [] -> []  
  | [x] -> []  
  | x :: y :: xs -> (x < y) :: foo (y :: xs)  
let bar = foo [1;6;2;3;5;4]
```

Justify your answers.

$\text{val foo} : \text{int list} \rightarrow \text{bool list}$

$\text{foo } [1;6;2;3;5;4] =$   
 $(1 < 6) :: \text{foo } [6;2;3;5;4] =$   
 $\text{true} :: (6 < 2) :: \text{foo } [2;3;5;4] =$   
 $\text{true} :: \text{false} :: (2 < 3) :: \text{foo } [3;5;4] =$   
 $\text{true} :: \text{false} :: \text{true} :: (3 < 5) :: \text{foo } [5;4] =$   
 $\text{true} :: \text{false} :: \text{true} :: \text{true} :: (5 < 4) :: \text{foo } [4] =$   
 $\text{true} :: \text{false} :: \text{true} :: \text{true} :: \text{false} :: []$

Note. This function returns the comparisons of adjacent elements

*This page is intentionally left blank.*